HYPERION AI

SYSTEM DESIGN DOCUMENT

Async Python Pipelines at Scale

Reference architecture for 19+ interlocked Python pipelines with Celery, FastAPI, Claude, and dedicated DLQ

PREPARED FOR

Teams running many async Python tools in parallel

Vitalijus Alsauskas

Software and AI Engineer

Hyperion AI - Production AI engineering for founders and agencies

https://hyperionai.dev · 2026-04-18

Async Python Pipelines at Scale - System Design

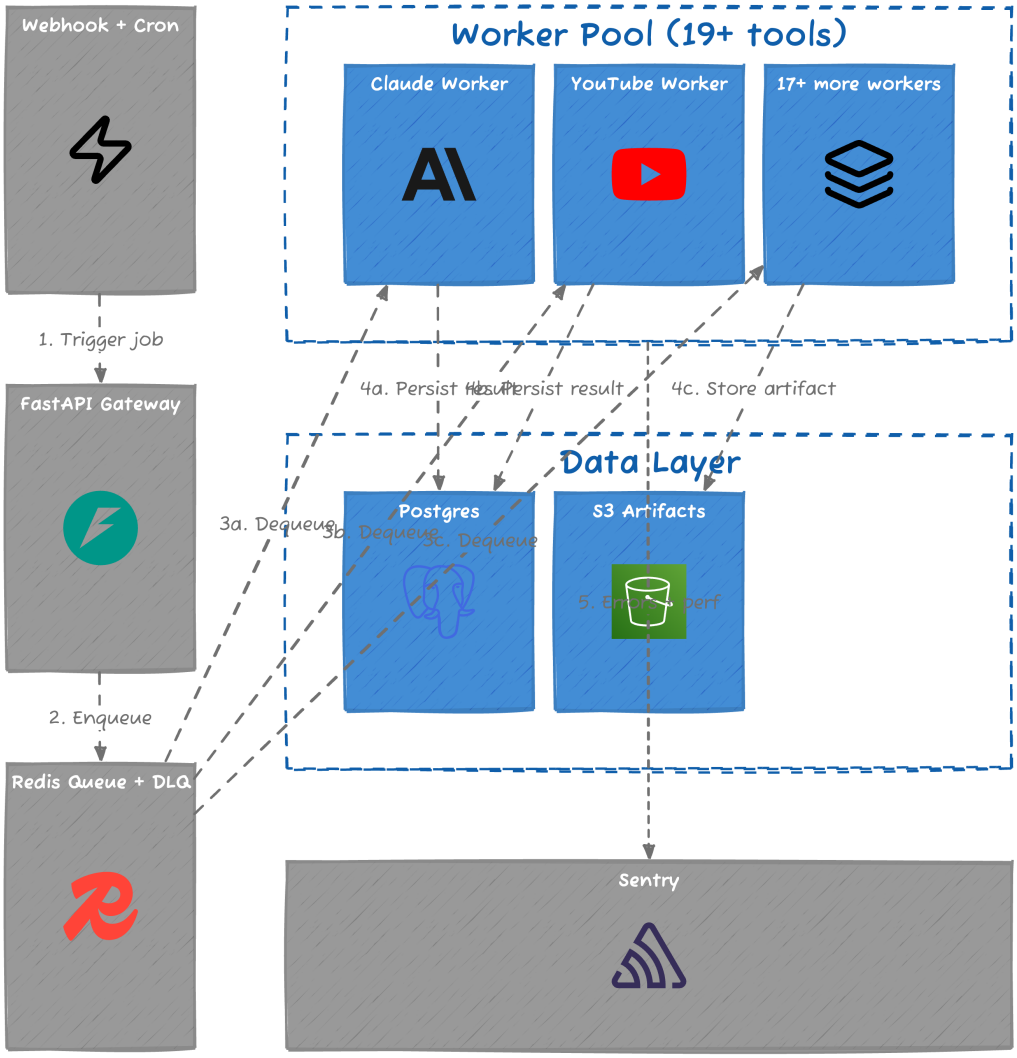
Problem Framing

Running 19+ interlocked Python pipelines in parallel is the point where most teams hit a wall. Each tool works in isolation on a laptop. In production, things compound: one worker starves the others, retries mask silent failures, logs are scattered across 19 streams, and when something breaks at 3am nobody knows which pipeline caused it.

This reference shows the pattern that survives: FastAPI gateway in front, Celery + Redis handling orchestration, workers isolated per tool, a dedicated DLQ so nothing silently dies, and structured logging with request IDs so one grep locates a job across all 19 workers.

Written for teams already running many Python pipelines who want to add one more without taking the platform down.

System Diagram



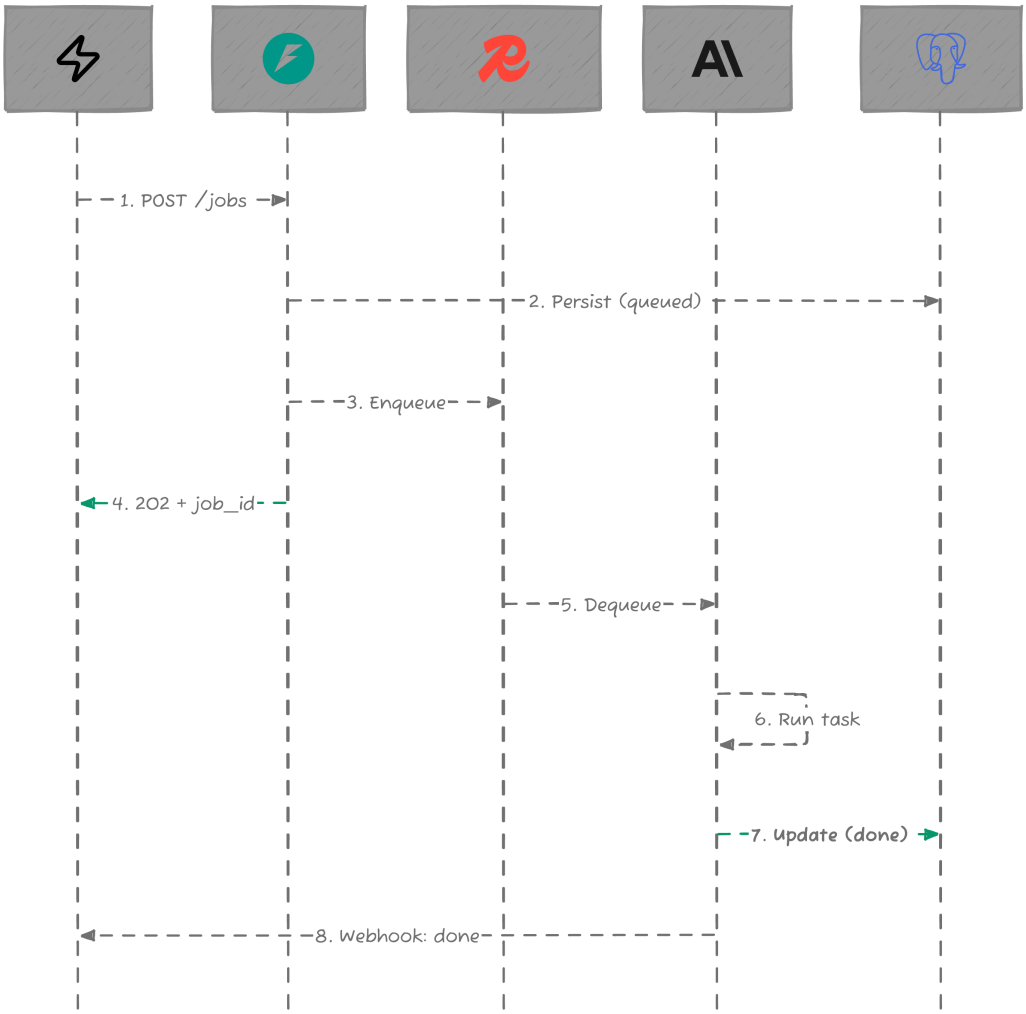
Source: `docs/architecture.d2`

Every pipeline tool runs as its own isolated Celery worker. A video render job failing never blocks a Claude extraction job. Every failed job that exhausts retries lands in a dedicated DLQ - never silently lost.

Tool Choices

LAYER	PICK	WHY THIS	ALTERNATIVES CONSIDERED
API Gateway	FastAPI + Pydantic	Async-native, OpenAPI auto, strong typing from day one	Flask (no async), Django (overkill for pure API), aiohttp (weaker DX)
Task Queue	Celery + Redis	Mature, battle-tested at this scale, fine-grained retry + DLQ control	RQ (simpler but weaker scheduling), Dramatiq (fine, smaller ecosystem), Arq (async-first but young)
Broker	Redis	Fast, cheap, already used for cache	RabbitMQ (more features, more ops burden), SQS (cloud lock-in)
Scheduler	Celery Beat	Co-located with Celery, one deploy	APScheduler (extra moving part), cron (no retry visibility)
Workers	Celery, one class per pipeline tool	Isolation, per-tool concurrency tuning	Monolithic worker (one bad job starves the rest), process pool (no retry machinery)
Observability	Structlog + Sentry + Flower	JSON logs, exception context, queue visibility	ELK stack (overkill), DataDog (expensive), print() statements (please no)
DB	Postgres	Job state + results, strong consistency	Mongo (worse for relational job graphs), Redis-only (no durability)
Object Storage	S3	Standard, cheap, Celery stores large artifacts here	Local disk (doesn't scale), GCS (fine, AWS is default here)
LLM	Anthropic Claude 3.5 Sonnet	Strong structured output, Claude Code workflow alignment	OpenAI (close but less consistent on structured JSON), Llama (local is cheaper but less reliable)

Data Flow



Source: `docs/flow.d2`

Happy path

1. Trigger fires (cron, webhook, or manual)
2. API validates and persists job row with `status=queued`
3. Job ID enqueued to Redis broker
4. Celery worker for the appropriate tool dequeues
5. Worker calls external APIs (Claude, YouTube, ffmpeg, etc.)
6. Artifacts land in S3, small results in Postgres

7. Job row updated to `status=done`
8. Outbound webhook fires to the downstream system

Error paths

- **Retryable failures** (rate limit, network timeout, 5xx): caught, logged to Sentry with context, requeued with exponential backoff up to the retry budget (default 3 retries)
- **Non-retryable failures** (malformed input, auth failure): marked failed immediately, logged, no retry (would just fail again)
- **Exhausted retries**: moved to DLQ, row updated to `status=failed` with full error context, Sentry critical alert, failure webhook fires. Nothing silently disappears.
- **Worker crash mid-job**: job ack only happens after successful commit, so a crashed worker means the job goes back on the queue automatically
- **DB write failure**: transaction rollback, worker marks job as failed-transient, retried

Component Breakdown

- **API Gateway** (FastAPI + Pydantic + structlog): public job submission, status, auth middleware, audit logging
- **Celery Beat**: scheduled job submission for cron-triggered pipelines
- **Worker Pool**: one Python module per pipeline tool under `workers/`, shared Celery app, each worker declares concurrency + retry + timeout in code
- **DLQ Monitor**: a lightweight service that reads the DLQ, fires Sentry alerts, and exposes a replay endpoint for ops
- **Observability Stack**: Structlog config shared across services, Sentry SDK for Python with async-aware instrumentation, Flower behind basic auth
- **Data Layer**: SQLAlchemy + Alembic for Postgres, boto3 for S3, redis-py for cache and broker access

Implementation Phases

Phase 1: Foundation (week 1-2)

Ships: Working skeleton handling one pipeline tool end-to-end with full retry + DLQ behavior.

- FastAPI gateway with one `/jobs` endpoint
- Celery + Redis + Beat wired up with docker-compose
- One worker class (say, the YouTube Data API worker) working end-to-end
- Structlog + Sentry + Flower all wired in
- DLQ monitor with alert on any job entering DLQ
- Integration tests covering happy path + retryable error + DLQ path

Phase 2: Pipeline Coverage (week 3-4)

Ships: All 19 pipeline tools migrated or wrapped into Celery workers.

- One worker class per tool, isolated
- Tool-specific retry + timeout policies tuned
- Load testing at 3x expected peak throughput
- Runbook: "what to do when worker X queue backs up"

Phase 3: Production Polish (week 5-6)

Ships: Observability refined, DLQ replay workflow, production deploy.

- Request ID propagation across all workers + external API calls
- DLQ replay UI or CLI (not just "look at Redis")
- Full runbook with per-worker playbook
- Claude Code `/test`, `/lint`, `/dev`, `/replay-dlq` slash commands
- Production deploy via GitHub Actions

Risk & Mitigation

1. **Risk:** A malformed input in one worker blocks the whole queue. **Mitigation:** Worker isolation means each tool has its own queue and worker pool. A broken worker cannot starve others. Each worker also declares a hard timeout so a hung external API cannot pin the pool indefinitely.
2. **Risk:** Retries mask silent failures - jobs disappear without anyone noticing. **Mitigation:** Dedicated DLQ. Every job that exhausts retries lands here, fires a Sentry critical alert, marks the job as `failed` in DB with full error context, and fires a failure webhook. Never silently lost.
3. **Risk:** 3am debugging requires reading 19 log streams. **Mitigation:** Structlog + request ID attached at ingress, propagated through every worker + external API call. One `grep request_id=abc123` surfaces the complete job history across all workers.
4. **Risk:** Claude API rate limits during peak throughput. **Mitigation:** Per-worker concurrency limits, exponential backoff on 429, a Claude-specific rate limiter in front of the LLM worker. Fallback to Claude Haiku under extreme load so the pipeline never fully stalls.
5. **Risk:** Production worker pool autoscaling misconfigured, jobs pile up at night. **Mitigation:** Queue-depth alerts in Sentry (warn at 100, critical at 500). Autoscaling rules tied to queue depth not CPU. Flower dashboard accessible from phone for on-call triage.

What I Need From You

- **Access to the existing codebase** (or a representative sample) so I can read what you have before proposing changes
- **A list of the 19+ tools** with a one-liner per tool on what each does
- **The current deployment target** (Hetzner VM? Container Apps? k8s? bare metal?)
- **Your current pain points in ranked order** - is 3am debugging the worst, or is it rate limits, or is it onboarding new pipelines?
- **A 30-minute call** to walk through this doc against your actual setup and tighten scope before any code ships

---

This document is yours to keep regardless of whether we end up working together. The architecture and tool choices here should save any developer who inherits this work a week of deliberation.